

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №20

Тема: Створення алгоритмів з розгалуженням

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Опанувати структуру «розгалуження»: умовні оператори if / elif / else, логічні операції та складні умови, вкладені розгалуження; навчитися будувати таблиці істинності, розробляти алгоритми з вибором варіанта дій та тестувати їх за критерієм покриття гілок і граничних значень.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal).

Короткі теоретичні відомості

1. Розгалужений алгоритм — алгоритм, у якому залежно від істинності певної умови виконується одна з кількох послідовностей дій. Розрізняють **неповне** розгалуження (дії виконуються лише за істинності умови) та **повне** (є гілка «інакше»).

2. Умовний оператор у Python:

```
if умова_1:
    дії_1
elif умова_2:
    дії_2
else:
    дії_3
```

Умови перевіряються **зверху вниз**; виконується перша гілка, умова якої істинна, а решта гілок пропускається. Блок else виконується, якщо жодна з умов не спрацювала. Тіло кожної гілки виділяється відступом (4 пробіли). Для простого вибору з двох значень існує умовний вираз: `m = a if a > b else b`.

3. Операції порівняння й логічні операції. Порівняння: `==`, `!=`, `<`, `<=`, `>`, `>=`. Логічні: `and` (І), `or` (АБО), `not` (НЕ). Пріоритет: `not` → `and` → `or`. У Python допустимі ланцюжкові порівняння: `0 <= x <= 100`. Обчислення `and/or` — **скорочене**: у виразі `b != 0 and a / b > 1` ділення не виконується, якщо `b = 0`.

A	B	A and B	A or B	not A
False	False	False	False	True
False	True	False	True	True
True	False	False	True	False
True	True	True	True	False

4. Закони де Моргана: `not (A and B) = (not A) or (not B)`; `not (A or B) = (not A) and (not B)`. Вони дозволяють спрощувати й перетворювати складні умови.

5. Тестування розгалужень. Щоб перевірити алгоритм із розгалуженням, потрібно забезпечити **покриття гілок** (кожна гілка виконується хоча б в одному тесті) та перевірити **граничні значення** — значення на межі умов і по обидва боки від неї. Наприклад, для умови `score >= 60` граничні тести — 59, 60, 61. Окремо перевіряють **некоректні вхідні дані** (від'ємні оцінки, нульові сторони).

6. Типові помилки: = замість ==; неправильний порядок умов в `elif` (більш загальна умова «перекриває» вужчу); пропущена гілка для граничного значення (> замість >=); надмірна вкладеність, яку можна замінити складною умовою або `elif`.

Хід роботи

Завдання 1. Таблиці істинності та закони де Моргана

Побудувати вручну таблиці істинності логічних виразів і перевірити еквівалентність пар виразів.

Порядок виконання:

1. Побудуйте таблицю істинності виразу $(A \text{ and not } B) \text{ or } C$ (8 рядків для всіх комбінацій A, B, C); додайте стовпці для проміжних значень `not B` та `A and not B`.
2. Перевірте за таблицею, що `not (a > 0 and b > 0)` еквівалентне `a <= 0 or b <= 0`; побудуйте таблицю для пар істинності $(a > 0, b > 0)$ і порівняйте стовпці.
3. Обчисліть значення виразів для $a = 5, b = -2, c = 0$: $a > 0 \text{ and } b > 0$, $a > 0 \text{ or } b > 0$, `not (c == 0)`, $a > b > c$, $a > 0 \text{ and } (b < 0 \text{ or } c < 0)$.
4. Запишіть за допомогою логічних операцій умови: (а) число x — двоцифрове; (б) числа a, b, c — довжини сторін трикутника; (в) рік y — високосний; (г) точка (x, y) лежить у першій чверті, але не на осях.
5. З'ясуйте, скільки різних наборів значень має таблиця істинності виразу з n змінних, і запишіть формулу.

Завдання 2. Трасування алгоритму класифікації трикутника

Виконати вручну алгоритм, що за трьома сторонами визначає, чи існує трикутник, і якого він виду.

```
алг Тип_трикутника
  ввести a, b, c
  якщо a + b <= c або a + c <= b або b + c <= a
    то вивести «не існує»
  інакше якщо a = b і b = c
    то вивести «рівносторонній»
  інакше якщо a = b або a = c або b = c
    то вивести «рівнобедрений»
  інакше вивести «різносторонній»
кін
```

Порядок виконання:

1. Накресліть блок-схему алгоритму: три ромби (перевірки) і чотири виходи з відповідями.
2. Побудуйте таблицю трасування зі стовпцями «a, b, c», «умова існування», «a = b = c», «є рівні сторони», «результат».
3. Виконайте алгоритм для наборів: (3, 4, 5), (5, 5, 5), (2, 2, 3), (1, 2, 3), (7, 7, 14), (0, 4, 4).
4. Поясніть результат для (1, 2, 3) і (7, 7, 14): чому нерівність у перевірці нестрога (`<=`)?

5. Доповніть алгоритм перевіркою прямокутності (теорема, обернена до теореми Піфагора): після визначення виду «різносторонній» або «рівнобедрений» перевірте, чи виконується $a^2 + b^2 = c^2$ для найбільшої сторони c .
6. Запишіть, скільки тестів потрібно мінімум для покриття всіх гілок алгоритму, і складіть їх.

Завдання 3. Програмна реалізація алгоритмів із розгалуженням

Реалізувати мовою Python класифікацію трикутника, розв'язання квадратного рівняння та перевірку високосного року; протестувати їх.

Порядок виконання:

1. Напишіть функцію `triangle_type(a, b, c)` за алгоритмом із завдання 2 (з додатковою перевіркою прямокутності). Вхідні дані, що не є додатними, вважайте некоректними.
2. Напишіть функцію `solve_quadratic(a, b, c)`: якщо $a = 0$ – розв'яжіть лінійне рівняння $bx + c = 0$ (окремо розгляньте $b = 0$); інакше обчисліть дискримінант $D = b^2 - 4ac$ і розгляньте три випадки ($D < 0$, $D = 0$, $D > 0$).
3. Напишіть функцію `is_leap(year)`: рік високосний, якщо він ділиться на 4, але не ділиться на 100, або ділиться на 400.
4. Напишіть функцію `max3(a, b, c)`, що знаходить найбільше з трьох чисел лише за допомогою розгалужень (без використання функції `max()`).
5. Складіть таблицю тестів і виконайте їх усі; виведіть для кожного тесту вхідні дані, результат і позначку ОК/ПОМИЛКА.

Приклад реалізації мовою Python:

python

```
import math

def triangle_type(a, b, c):
    if a <= 0 or b <= 0 or c <= 0:
        return "некоректні дані"
    if a + b <= c or a + c <= b or b + c <= a:
        return "не існує"
    if a == b == c:
        return "рівносторонній"
    x, y, z = sorted((a, b, c))
    right = "прямокутний" if x * x + y * y == z * z else ""
    if a == b or a == c or b == c:
        return "рівнобедрений" + right
    return "різносторонній" + right

def solve_quadratic(a, b, c):
    if a == 0:
        if b == 0:
            return "розв'язків немає" if c != 0 else "будь-яке x"
            return f"x = {-c / b:g}"
        d = b * b - 4 * a * c
        if d < 0:
            return "дійсних коренів немає"
        if d == 0:
            return f"x = {-b / (2 * a):g}"
        x1 = (-b - math.sqrt(d)) / (2 * a)
        x2 = (-b + math.sqrt(d)) / (2 * a)
        return f"x1 = {x1:g}, x2 = {x2:g}"

def is_leap(year):
    return (year % 4 == 0 and year % 100 != 0) or year % 400 == 0
```

```
def max3(a, b, c):
    m = a
    if b > m:
        m = b
    if c > m:
        m = c
    return m

for t in ((3, 4, 5), (5, 5, 5), (2, 2, 3), (1, 2, 3), (7, 7, 14), (0, 4, 4), (6, 7, 8)):
    print(f"triangle_type{t} -> {triangle_type(*t)}")

for q in ((1, -5, 6), (1, -4, 4), (1, 2, 5), (0, 2, -8), (0, 0, 3)):
    print(f"{q[0]}x^2 + {q[1]}x + {q[2]} = 0 -> {solve_quadratic(*q)}")

print("Високосні:", [y for y in (1900, 2000, 2023, 2024, 2100) if is_leap(y)])
print("max3:", max3(4, 9, 2), max3(-1, -5, -3), max3(7, 7, 7))
```

Результат виконання програми:

```
triangle_type(3, 4, 5) -> різносторонній прямокутний
triangle_type(5, 5, 5) -> рівносторонній
triangle_type(2, 2, 3) -> рівнобедрений
triangle_type(1, 2, 3) -> не існує
triangle_type(7, 7, 14) -> не існує
triangle_type(0, 4, 4) -> некоректні дані
triangle_type(6, 7, 8) -> різносторонній
1x^2 + -5x + 6 = 0 -> x1 = 2, x2 = 3
1x^2 + -4x + 4 = 0 -> x = 2
1x^2 + 2x + 5 = 0 -> дійсних коренів немає
0x^2 + 2x + -8 = 0 -> x = 4
0x^2 + 0x + 3 = 0 -> розв'язків немає
Високосні: [2000, 2024]
max3: 9 -1 7
```

Висновок за результатами: розгалуження коректно розрізняють усі гілки: (3, 4, 5) — прямокутний різносторонній трикутник, (1, 2, 3) і (7, 7, 14) — не існують (нерівність нестрога), (0, 4, 4) — некоректні дані. Квадратне рівняння має три різні результати залежно від знака дискримінанта, а також окремо оброблені вироджені випадки $a = 0$. З років 1900, 2000, 2023, 2024, 2100 високосними є лише 2000 і 2024 — 1900 та 2100 діляться на 100, але не на 400.

Завдання 4. Тестування за граничними значеннями та порядок умов

Реалізувати перекладання балів за шкалою ECTS, протестувати її на граничних значеннях і продемонструвати помилку неправильного порядку умов в elif.

Порядок виконання:

1. Для шкали: 90–100 – «А», 82–89 – «В», 74–81 – «С», 64–73 – «D», 60–63 – «Е», 35–59 – «FХ», 1–34 – «F» складіть список граничних значень (по два на кожен межу: 34 і 35, 59 і 60, 63 і 64 тощо) та некоректних значень (–1, 0, 101).
2. Напишіть функцію `ects(score)` на ланцюжку `if ... elif ... else` з перевіркою коректності ($0 < score \leq 100$), використовуючи ланцюжкові порівняння.

3. Напишіть «помилкову» версію `ects_buggy(score)`, у якій умови розташовано в іншому порядку (`score >= 60` — першою), і порівняйте результати обох версій на всіх тестових значеннях.
4. Виведіть таблицю «score — правильна оцінка — помилкова версія» лише для тих значень, де результати розбігаються, і підрахуйте кількість помилкових відповідей.
5. Поясніть, чому перевірка умов у порядку від найвужчої до найширшої (або навпаки) впливає на результат, і сформулюйте правило складання ланцюжків `elif`.

Приклад реалізації мовою Python:

python

```
def ects(score):
    if not 0 < score <= 100:
        return "некоректно"
    if score >= 90:
        return "A"
    elif score >= 82:
        return "B"
    elif score >= 74:
        return "C"
    elif score >= 64:
        return "D"
    elif score >= 60:
        return "E"
    elif score >= 35:
        return "FX"
    else:
        return "F"

def ects_buggy(score):
    if not 0 < score <= 100:
        return "некоректно"
    if score >= 60:
        return "E"
    elif score >= 64:
        return "D"
    elif score >= 74:
        return "C"
    elif score >= 82:
        return "B"
    elif score >= 90:
        return "A"
    elif score >= 35:
        return "FX"
    else:
        return "F"

tests = [-1, 0, 1, 34, 35, 59, 60, 63, 64, 73, 74, 81, 82, 89, 90, 100, 101]
print("score    правильно помилкова версія")
wrong = 0
for s in tests:
    good, bad = ects(s), ects_buggy(s)
    if good != bad:
        wrong += 1
        print(f"{s:>5}    {good:>9}    {bad:>16}")
print(f"Розбіжностей: {wrong} із {len(tests)} тестів")
```

Результат виконання програми:

score	правильно	помилкова версія
64	D	E
73	D	E
74	C	E
81	C	E
82	B	E
89	B	E

90	A	E
100	A	E

Розбіжностей: 8 із 17 тестів

Висновок за результатами: правильна версія видає очікувані оцінки на всіх граничних значеннях, а помилкова версія з умовою `score >= 60` на першому місці повертає «E» для 8 зі 17 тестів — будь-яка оцінка від 60 і вище «перехоплюється» першою, найширшою умовою, а наступні гілки стають недосяжними. Звідси правило: в ланцюжку `elif` умови розташовують так, щоб спочатку перевірялися вузьчі (найсуворіші) умови, а загальніші — пізніше; обов'язково тестують значення на межах інтервалів і по обидва боки від них.

Контрольні питання

1. Який алгоритм називається розгалуженням? Чим відрізняється повне розгалуження від неповного?
2. Опишіть синтаксис і порядок виконання умовного оператора `if ... elif ... else` в Python.
3. Назвіть логічні операції та їхній пріоритет. Що таке скорочене обчислення логічних виразів?
4. Сформулюйте закони де Моргана та наведіть приклад їх застосування до умови.
5. Що таке ланцюжкове порівняння і чим воно зручне?
6. Як перевірити, чи рік високосний? Чому 1900 не є високосним, а 2000 — є?
7. Що означає покриття гілок і граничні значення при тестуванні алгоритмів із розгалуженням?
8. До яких наслідків призводить неправильний порядок умов в ланцюжку `elif`? Наведіть приклад.

Зміст звіту

1. Тема, мета роботи.
2. Таблиці істинності (завдання 1), блок-схема та таблиця трасування (завдання 2).
3. Тексти програм, таблиця тестів та результати їх роботи (завдання 3, 4).
4. Висновки.